

HANDS-ON 2 [Beginner]

Django Models, ORM & Admin Interface

Topics Covered:

Django Models✓

Field Types & Constraints✓

Migrations✓

Django ORM Queries✓

Admin Interface Registration✓

You will now define the data models for the Course Management system, run migrations to create database tables, query data using the Django ORM, and explore the built-in admin interface.

Task 1: Define Models and Run Migrations

Goal: Create Django models that represent the Course Management domain.

Steps:

11.

In `courses/models.py`, define the following models: `Department` (`name`, `head_of_dept`, `budget`), `Course` (`name`, `code` — unique, `credits`, `department` — `ForeignKey`), `Student` (`first_name`, `last_name`, `email` — unique, `department` — `ForeignKey`, `enrollment_year`), `Enrollment` (`student` — `ForeignKey`, `course` — `ForeignKey`, `enrollment_date`, `grade` — nullable `CharField`).

12.

Add `__str__` methods to each model returning a human-readable string (e.g., return `self.name` for `Course`).

13.

Run `python manage.py makemigrations` to generate the migration files.

Inspect the generated migration file — identify the `CreateTable` operations.

14.

Run `python manage.py migrate` to apply migrations. Confirm the tables exist by checking the database with `python manage.py dbshell`.

15.

Add a Meta class to the `Enrollment` model with `unique_together = [['student', 'course']]` to prevent duplicate enrollments.

Hint:

-

`ForeignKey(Model, on_delete=models.CASCADE)` means deleting a `Department` also deletes all its `Courses` — choose `on_delete` carefully based on business rules.

-

Always run `makemigrations` after changing models, then `migrate` to apply — never edit migration files manually unless you know what you are doing.

Expected Outcome: `python manage.py showmigrations` shows all migrations as applied [X]. The `dbshell` shows the created tables.

Task 2: Django ORM Queries

Goal: Use the Django ORM shell to perform CRUD operations and complex queries.

Steps:

16.

Open the Django shell with `python manage.py shell`. Create at least 2 Department, 4 Course, and 5 Student objects using `Model.objects.create(...)`.

17.

Query all courses in a specific department using `Course.objects.filter(department__name='Computer Science')`. Note the double underscore — this is a Django ORM lookup across a ForeignKey.

18.

Use `.values()` and `.annotate()` to count the number of courses per department: `Department.objects.annotate(course_count=Count('course'))`.

19.

Use `select_related` to fetch all students along with their department in a single SQL query. Confirm with Django's `connection.queries` log.

20.

Perform an update: increase the budget of all departments by 10% using `Department.objects.update(budget=F('budget') * 1.1)`.

Hint:

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Python Backend Frameworks | Hands-On Exercise Book

•

The double underscore (`__`) is Django ORM's way of traversing relationships and applying lookups — e.g., `department__name` performs a JOIN.

•

`F()` objects let you reference field values in expressions without pulling data into Python — the calculation happens in the database.

Expected Outcome: Shell queries return expected results. The `annotate` query shows course counts per department. The `F()` update runs without fetching rows into Python.

Task 3: Django Admin Interface

Goal: Register models with the Django admin and explore its built-in management features.

Steps:

21.

Create a superuser: `python manage.py createsuperuser`. Use `admin / admin@college.edu / Admin@123` for local testing.

22.

In `courses/admin.py`, register all four models: `admin.site.register(Department)`, etc. Start the server and explore the admin at `/admin/`.

23.

Customise the `CourseAdmin`: use `list_display = ['name', 'code', 'credits', 'department']` to show multiple columns in the list view. Add `search_fields = ['name', 'code']` to enable search.

24.

Add `list_filter = ['department']` to `CourseAdmin` to enable sidebar filtering by department.

25.

Create 3 courses, 5 students, and 4 enrollments through the admin interface. Verify the `unique_together` constraint on Enrollment by trying to enroll the same student in the same course twice.

Hint:

-

The admin interface is Django's superpower for rapid prototyping — you get full CRUD for all models with minimal code.

-

ModelAdmin's `list_display`, `search_fields`, and `list_filter` are the three most impactful customisations for everyday use.

Expected Outcome: Admin lists courses with name, code, credits, department columns. Search and filter work. Duplicate enrollment raises a validation error.

HANDS-ON 2 [Beginner]

Django Models, ORM & Admin Interface

Topics Covered:

Django Models✓

Field Types & Constraints✓

Migrations✓

Django ORM Queries✓

Admin Interface Registration✓

You will now define the data models for the Course Management system, run migrations to create database tables, query data using the Django ORM, and explore the built-in admin interface.

Task 1: Define Models and Run Migrations

Goal: Create Django models that represent the Course Management domain.

Steps:

11.

In `courses/models.py`, define the following models: Department (name, head_of_dept, budget), Course (name, code — unique, credits, department — ForeignKey), Student (first_name, last_name, email — unique, department — ForeignKey, enrollment_year), Enrollment (student — ForeignKey, course — ForeignKey, enrollment_date, grade — nullable CharField).

12.

Add `__str__` methods to each model returning a human-readable string (e.g., return `self.name` for Course).

13.

Run `python manage.py makemigrations` to generate the migration files.

Inspect the generated migration file — identify the CreateTable operations.

14.

Run `python manage.py migrate` to apply migrations. Confirm the tables exist by checking the database with `python manage.py dbshell`.

15.

Add a Meta class to the Enrollment model with `unique_together = [['student', 'course']]` to prevent duplicate enrollments.

Hint:

-

`ForeignKey(Model, on_delete=models.CASCADE)` means deleting a Department also deletes all its Courses — choose `on_delete` carefully based on business rules.

-

Always run `makemigrations` after changing models, then `migrate` to apply — never edit migration files manually unless you know what you are doing.
Expected Outcome: `python manage.py showmigrations` shows all migrations as applied [X]. The `dbshell` shows the created tables.

Task 2: Django ORM Queries

Goal: Use the Django ORM shell to perform CRUD operations and complex queries.

Steps:

16.

Open the Django shell with `python manage.py shell`. Create at least 2 Department, 4 Course, and 5 Student objects using `Model.objects.create(...)`.

17.

Query all courses in a specific department using `Course.objects.filter(department__name='Computer Science')`. Note the double underscore — this is a Django ORM lookup across a `ForeignKey`.

18.

Use `.values()` and `.annotate()` to count the number of courses per department: `Department.objects.annotate(course_count=Count('course'))`.

19.

Use `select_related` to fetch all students along with their department in a single SQL query. Confirm with Django's `connection.queries` log.

20.

Perform an update: increase the budget of all departments by 10% using `Department.objects.update(budget=F('budget') * 1.1)`.

Hint:

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Python Backend Frameworks | Hands-On Exercise Book

-

The double underscore (`__`) is Django ORM's way of traversing relationships and applying lookups — e.g., `department__name` performs a JOIN.

-

`F()` objects let you reference field values in expressions without pulling data into Python — the calculation happens in the database.

Expected Outcome: Shell queries return expected results. The `annotate` query shows course counts per department. The `F()` update runs without fetching rows into Python.

Task 3: Django Admin Interface

Goal: Register models with the Django admin and explore its built-in management features.

Steps:

21.

Create a superuser: `python manage.py createsuperuser`. Use `admin / admin@college.edu / Admin@123` for local testing.

22.

In `courses/admin.py`, register all four models: `admin.site.register(Department)`, etc. Start the server and explore the admin at `/admin/`.

23.

Customise the `CourseAdmin`: use `list_display = ['name', 'code', 'credits', 'department']` to show multiple columns in the list view. Add `search_fields = ['name', 'code']` to enable search.

24.

Add `list_filter = ['department']` to `CourseAdmin` to enable sidebar filtering by department.

25.

Create 3 courses, 5 students, and 4 enrollments through the admin interface. Verify the `unique_together` constraint on `Enrollment` by trying to enroll the same student in the same course twice.

Hint:

-

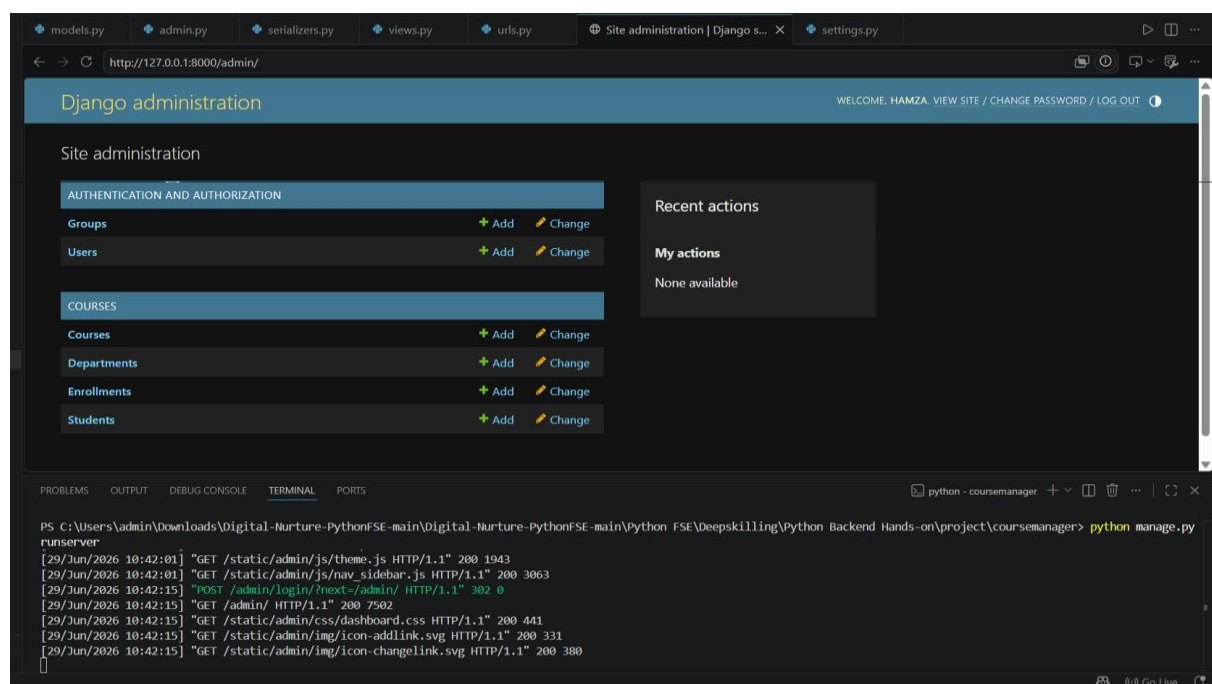
The admin interface is Django's superpower for rapid prototyping — you get full CRUD for all models with minimal code.

-

`ModelAdmin`'s `list_display`, `search_fields`, and `list_filter` are the three most impactful customisations for everyday use.

Expected Outcome: Admin lists courses with name, code, credits, department columns. Search and filter work. Duplicate enrollment raises a validation error.

SOLUTION:



models.pyadmin.pyserializers.pyviews.pyurls.pySelect course to change | Djan... ×settings.py

← → http://127.0.0.1:8000/admin/courses/course/

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

COURSES

Courses + Add

Departments + Add

Enrollments + Add

Students + Add

Select course to change

ADD COURSE +

Action: ----- Go 0 of 5 selected

☐	NAME	CODE	DEPARTMENT
☐	Introduction to Machine Learning	19AI45	AIML (raj)
☐	Hydraulic Systems	19MC21	Mech (manoj)
☐	Environmental Studies	19EC23	Civil (manoj)
☐	Data Structures	19AI550	AIDS (raj)
☐	Cloud Computing	19cs550	CSE (abc)

5 courses

FILTER

Show counts

↓ By department

All

CSE (abc)

AIDS (raj)

AIML (raj)

Cyber Security (xyz)

Civil (manoj)

Mech (manoj)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

python - coursemanager + - - - - -

PS C:\Users\admin\Downloads\Digital-Nurture-PythonFSE-main\Digital-Nurture-PythonFSE-main\Python FSE\Deepskilling\Python Backend Hands-on\project\coursemanager> python manage.py runserver

[29/Jun/2026 10:42:55] "GET /static/admin/js/prepopulate.js HTTP/1.1" 200 1531

[29/Jun/2026 10:42:55] "GET /static/admin/js/vendor/jquery/jquery.js HTTP/1.1" 200 285314

[29/Jun/2026 10:42:55] "GET /static/admin/img/search.svg HTTP/1.1" 200 458

[29/Jun/2026 10:42:55] "GET /static/admin/js/filters.js HTTP/1.1" 200 978

[29/Jun/2026 10:42:55] "GET /static/admin/js/vendor/xregexp/xregexp.js HTTP/1.1" 200 325171

[29/Jun/2026 10:42:55] "GET /static/admin/img/tooltag-add.svg HTTP/1.1" 200 331

[29/Jun/2026 10:42:55] "GET /static/admin/img/icon-viewlink.svg HTTP/1.1" 200 581